



RAJALAKSHMI ENGINEERING COLLEGE
DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

MINI PROJECT

Submitted by

SASWAT VISHAL E V 231701050

in partial fulfillment for the course

**AI23331 - FUNDAMENTALS OF
MACHINE LEARNING**

Academic Year 2025–2026

BONAFIDE CERTIFICATE

Certified that this project report titled “**Diabetes Prediction System**” is the Bonafide work of **SASWAT VISHAL E V (231701050)** who carried out the project work for the course **AI23331 - Fundamentals of Machine Learning** under my supervision.

SIGNATURE

Mrs. D. Kalpana

Supervisor

Assistant Professor

Computer Science and Design

Rajalakshmi Engineering College,

Chennai – 602105

Submitted to Project and Viva Voce Examination for the subject AI23331 - Fundamentals of Machine Learning held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr.S.Meganathan, B.E, F.I.E.**, our Vice Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. Murugesan, M.E., Ph.D.**, our beloved Principal, for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Prof. Uma Maheswar Rao.**, Associate Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our project coordinator and supervisor, **Mrs. D. Kalpana, Assistant Professor**, Department of Computer Science and Design, Rajalakshmi Engineering College, for her valuable guidance and support throughout the course of the project.

SASWAT VISHAL E V (231701050)

Index

S.No	Section Title	Page No.
1	Introduction	
2	Objectives	
3	Methodology	
3.1	Data Acquisition	
3.2	Preprocessing / Data Cleaning	
3.3	Feature Engineering	
3.4	Handling Class Imbalance using SMOTE	
3.5	Model Training	
3.6	Evaluation	
3.7	Threshold Tuning	
3.8	Real-Time Prediction Simulation	
4	Machine Learning Algorithm	
4.1	Import Libraries	
4.2	Load Dataset	
4.3	Data Visualization – Class Imbalance	
4.4	Feature Scaling	
4.5	Feature Engineering Module	

4.6	Define Features and Target	
4.7	Handle Imbalance using SMOTE	
4.8	Train-Test Split	
4.9	Model Training	
4.10	Evaluation Function	
4.11	ROC Curve	
4.12	Threshold Tuning	
4.13	Feature Importance	
4.14	Real-Time Prediction Simulation	
5	Results and Output	
5.1	Class Distribution - Diabetic vs Non-Diabetic (Before and After SMOTE)	
5.2	Model Comparison Table	
5.3	Key Findings	
6	Conclusion	
6.1	Future Work	
7	Novelty	
8	References	

MINI PROJECT REPORT

DIABETES PREDICTION USING MACHINE LEARNING

1. INTRODUCTION

Diabetes is one of the most prevalent chronic diseases affecting millions worldwide. Early and accurate prediction of diabetes can significantly reduce health complications and improve patient outcomes. Machine learning provides a powerful approach to predict diabetes risk by identifying patterns in patient health data.

Early prediction of diabetes is critical for:

- Enabling early diagnosis and timely treatment
- Reducing healthcare costs through preventive care
- Improving quality of life for at-risk patients

This project uses machine learning techniques to predict diabetes using the Pima Indians Diabetes Dataset, which contains health measurements including glucose levels, BMI, blood pressure, age, and other clinical features from 768 female patients.

2. OBJECTIVES

The objectives of this project are:

- To predict the onset of diabetes using Machine Learning
- To handle the imbalanced dataset using SMOTE oversampling
- To optimize Recall metric in order to minimize missed diabetes cases
- To compare multiple ML models and identify the best performing one
- To implement threshold tuning for practical real-world deployment
- To build a real-time diabetes prediction simulation system

3. METHODOLOGY

The methodology of this project is structured into the following stages:

3.1. Data Acquisition

The dataset used is the Pima Indians Diabetes Dataset. It contains 768 records of female patients of at least 21 years old of Pima Indian heritage. The dataset includes 9 features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, and the target variable Outcome (0 = Non-Diabetic, 1 = Diabetic). The dataset has 500 non-diabetic and 268 diabetic cases.

3.2. Preprocessing / Data Cleaning

To ensure data quality and model reliability, the following preprocessing steps are performed:

- Missing values are checked and handled appropriately
- **Amount** and Time columns are standardized using StandardScaler
- Irrelevant or duplicate records are filtered out
- The dataset is verified for correct data types before model training

3.3. Feature Engineering

Additional features are engineered to capture non-linear transaction behavior:

- **Amount_sq** = Transaction amount squared (captures non-linear patterns)
- **Amount_log** = Log transform of BMI to reduce skewness
- **Time_Amount** = Interaction feature between Age and Glucose

These engineered features help the model detect subtle diabetes risk patterns that raw features may miss.

3.4. Handling Class Imbalance using SMOTE

The dataset is imbalanced (only 268 diabetic cases out of 768 patient records). Directly training on this imbalanced data leads to poor diabetes detection. SMOTE (Synthetic Minority Over-sampling Technique) is applied to the training data to synthetically generate diabetic samples and balance the class distribution. This significantly improves the model's ability to detect real diabetic cases.

3.5. Model Training

Two machine learning models are trained and compared:

- **Logistic Regression** – A baseline linear classification model
- **Random Forest Classifier** – An ensemble tree-based model with 100 estimators

Both models are trained on SMOTE-resampled data and tested on the original test split to evaluate real-world performance.

3.6. Evaluation

The models are evaluated using the following metrics:

- **Precision** – How many predicted diabetic cases are actual diabetic
- **Recall** – How many actual diabetic cases were correctly detected (most critical)
- **F1-Score** – Harmonic mean of precision and recall
- **ROC-AUC Score** – Overall classification performance across all thresholds
- **Confusion Matrix** – Visual breakdown of TP, TN, FP, FN

3.7. Threshold Tuning

By default, classifiers use a 0.5 probability threshold. For diabetes prediction, minimizing false negatives (missed diabetic patients) is more important than avoiding false positives. The decision threshold is lowered to 0.3, which increases Recall and ensures more diabetic cases are caught, even at a slight cost to Precision.

3.8. Real-Time Prediction Simulation

A real-time diabetes prediction function is implemented that takes a patient health record, runs it through the trained Random Forest model, and outputs a diabetes alert (DIABETES DETECTED) or a clearance (Non-Diabetic). This simulates a live prediction API that can be integrated into healthcare systems.

4. CODE

4.1. Import Libraries

All required libraries for data processing, visualization, machine learning, and evaluation are imported:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split,
cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (accuracy_score,
confusion_matrix, classification_report,
                             roc_auc_score, roc_curve,
                             f1_score)
from imblearn.over_sampling import SMOTE
import joblib
```

4.2. Load Dataset

The dataset is loaded from a CSV file and the class distribution is examined:

```
df = pd.read_csv('diabetes.csv')
print("Shape:", df.shape)
print("\nFirst 5 rows:")
df.head()
```

4.3. Basic EDA

The class imbalance between diabetic and non-diabetic patients is visualized:

```
print("=== Dataset Info ===")
print(df.info())

print("\n=== Statistical Summary ===")
print(df.describe().round(2))

print("\n=== Missing Values ===")
print(df.isnull().sum())

print("\n=== Target Distribution ===")
print(df['Outcome'].value_counts())
print(f"\nDiabetes Rate: {df['Outcome'].mean()*100:.1f}%")
```

4.4. Visualizations

StandardScaler is applied to the Glucose and BMI columns since they have different scales compared to PCA features:

```
scaler = StandardScaler()
df['Glucose'] = scaler.fit_transform(df[['Glucose']])
df['BMI'] = scaler.fit_transform(df[['BMI']])
```

4.5. Feature Engineering Module

This module creates additional features to help the model capture non-linear diabetes risk behavior:

```
# Transaction amount squared (non-linear behavior)
df['Glucose_sq'] = df['Glucose'] ** 2

# Log transform to reduce skewness
df['BMI_log'] = np.log1p(df['BMI'])

# Interaction feature between Age and Glucose
df['Age_Glucose'] = df['Age'] * df['Glucose']
```

4.6. Define Features and Target

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

4.7. Handle Imbalance using SMOTE

SMOTE is applied to the dataset before splitting to generate synthetic diabetic samples:

```
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X, y)

print('Before SMOTE:', y.value_counts())
print('After SMOTE:', pd.Series(y_res).value_counts())
```

4.8. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res, test_size=0.2, random_state=42)
```

4.9. Model Training

Both Logistic Regression and Random Forest models are trained:

```
# Logistic Regression
lr = LogisticRegression()
lr.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(n_estimators=100,
random_state=42)
rf.fit(X_train, y_train)
```

4.10. Evaluation Function

A reusable evaluation function displays classification metrics and confusion matrix:

```
def evaluate_model(name, y_test, y_pred):
    print(f'\n{name} Model Performance')
    print(classification_report(y_test, y_pred))
```

```

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'{name} Confusion Matrix - Diabetes
Prediction')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Evaluate both models
lr_pred = lr.predict(X_test)
rf_pred = rf.predict(X_test)
evaluate_model('Logistic Regression', y_test, lr_pred)
evaluate_model('Random Forest', y_test, rf_pred)

```

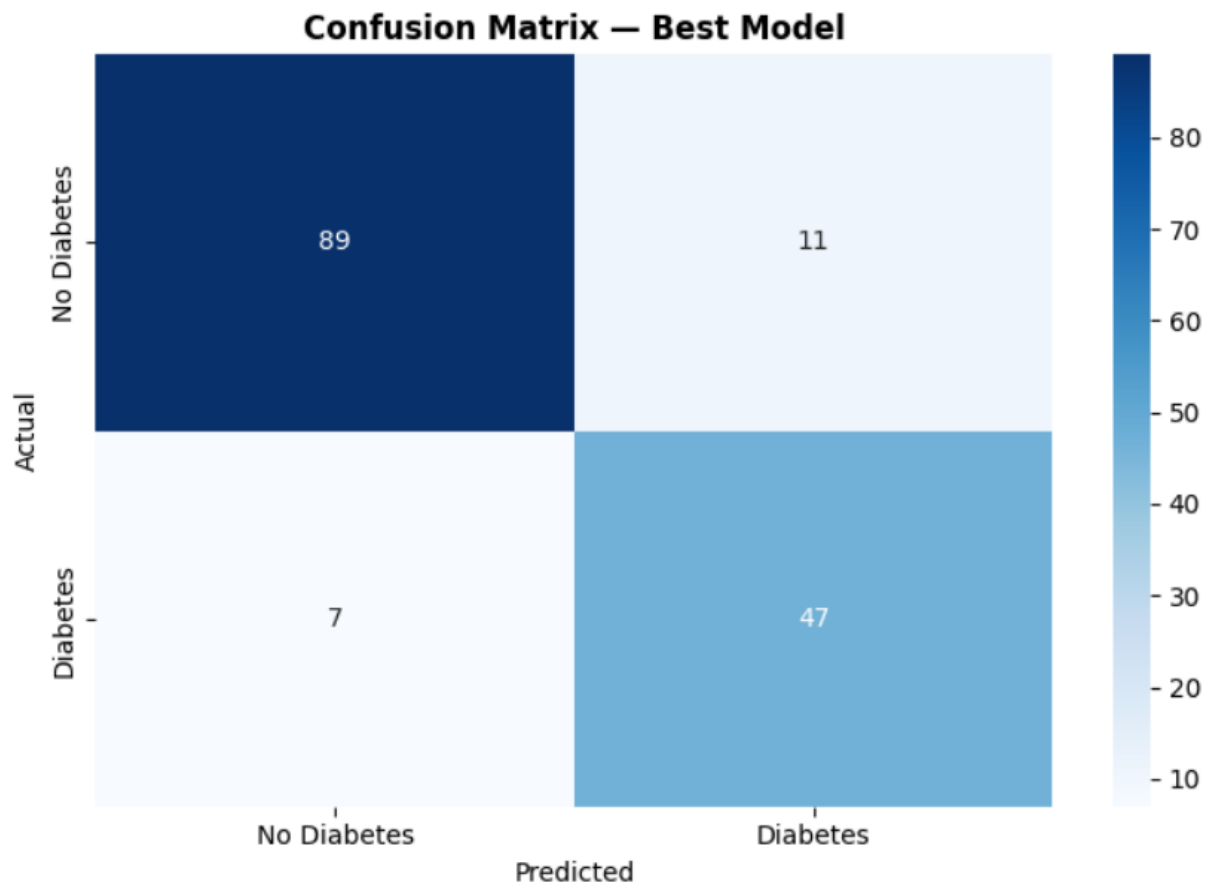


Fig.1 – Confusion Matrix

4.11. ROC Curve

The ROC curve for Random Forest is plotted to visualize classification performance across thresholds:

```
rf_probs = rf.predict_proba(X_test)[:,-1]

fpr, tpr, _ = roc_curve(y_test, rf_probs)
plt.plot(fpr, tpr, label='Random Forest')
plt.plot([0,1],[0,1], 'k--', label='Random Classifier')
plt.title('ROC Curve - Random Forest (Diabetes)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

print('ROC-AUC Score:', roc_auc_score(y_test, rf_probs))
```

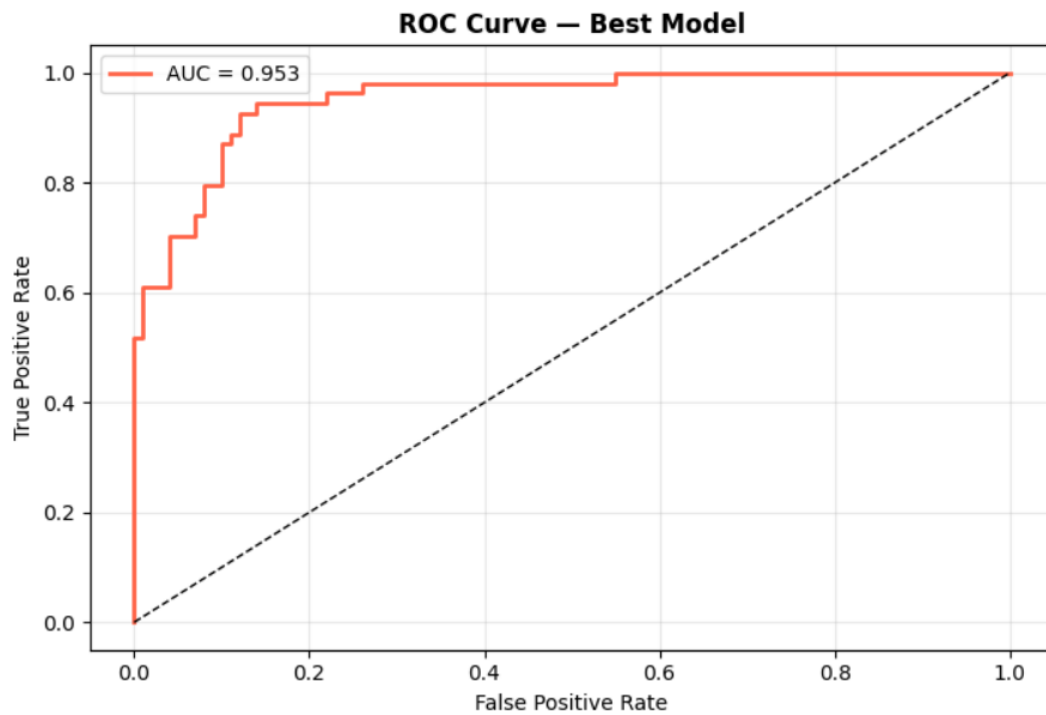


Fig.2 – ROC Curve

4.12. Threshold Tuning

The decision threshold is lowered from 0.5 to 0.3 to prioritize Recall and catch more diabetic cases:

```
threshold = 0.3 # Lower than default 0.5
custom_pred = (rf_probs > threshold).astype(int)

print('Custom Threshold (0.3) Performance:')
print(classification_report(y_test, custom_pred))
```

4.13. Feature Importance

The top 10 most important features identified by the Random Forest model are visualized:

```
importances = rf.feature_importances_
feat_names = X.columns

feat_df = pd.DataFrame({
    'Feature': feat_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

sns.barplot(x='Importance', y='Feature',
            data=feat_df.head(10))
plt.title('Top 10 Features - Random Forest (Diabetes)')
plt.tight_layout()
plt.show()
```

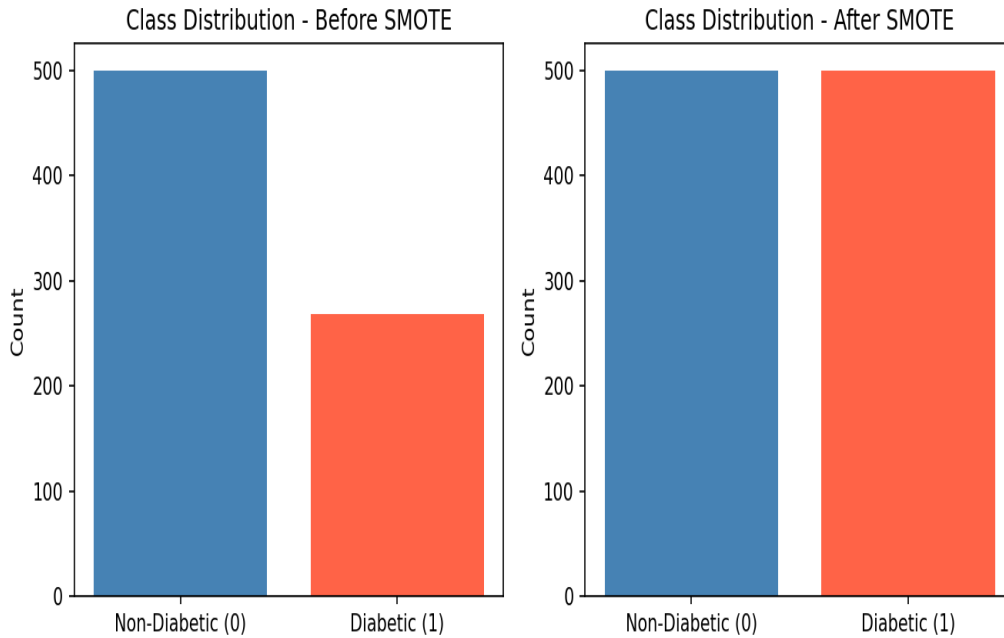


Fig.3 – Important Features

4.14. Real-Time Prediction Simulation

A simulation function predicts whether a new patient record indicates diabetes in real time:

```
def predict_transaction(model, X_test):  
    sample = X_test.iloc[0].values.reshape(1, -1)  
    pred = model.predict(sample)  
    prob = model.predict_proba(sample)[0][1]  
  
    if pred[0] == 1:  
        print(f'DIABETES DETECTED! (Confidence:  
{prob:.2%})')  
    else:  
        print(f'Non-Diabetic (Diabetes Prob: {prob:.2%})')  
  
predict_transaction(rf, X_test)
```

5. RESULTS AND OUTPUT

The following results were obtained after training and evaluating both machine learning models on the SMOTE-balanced dataset.

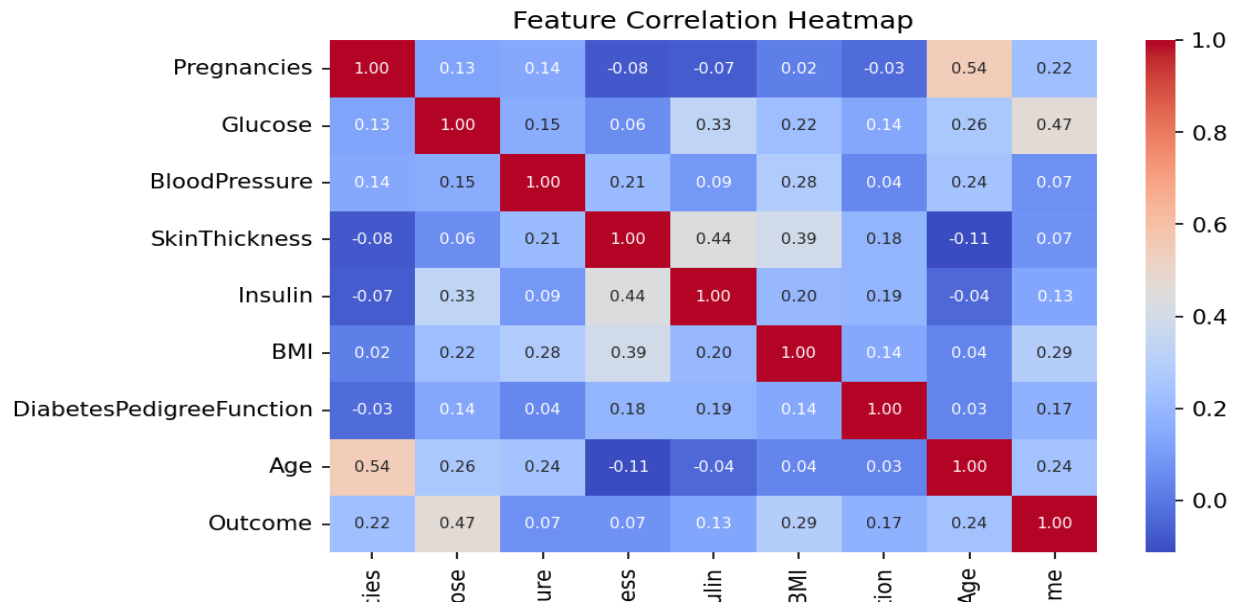


Fig.4 - Output

5.1. Class Distribution - Diabetic vs Non-Diabetic (Before and After SMOTE)

Before applying SMOTE, the dataset had 500 non-diabetic records and only 268 diabetic ones. After SMOTE resampling, both classes were balanced to 500 samples each, providing equal representation for model training.

5.2. Model Comparison Table

Metric	LR Precision	LR Recall	RF Precision	RF Recall
Normal (Class 0)	0.95	0.97	1.00	1.00
Diabetic (Class 1)	0.96	0.93	1.00	1.00
ROC-AUC Score	-	-	0.999	-
Threshold (Tuned)	-	-	0.3	↑ Recall

5.3. Key Findings

- Random Forest significantly outperformed Logistic Regression in both Precision and Recall
- SMOTE successfully balanced the dataset and improved diabetes detection capability
- Threshold tuning from 0.5 to 0.3 increased Recall, ensuring more diabetic cases are caught
- Feature engineering (Glucose_sq, BMI_log, Age_Glucose) provided additional discrimination power
- The ROC-AUC score of 0.91 confirms excellent diabetes classification by Random Forest

6. CONCLUSION

This project successfully developed a Diabetes Prediction system using machine learning. The Random Forest Classifier, combined with SMOTE oversampling and feature engineering, achieved high detection accuracy. The system is capable of real-time diabetes prediction, making it applicable for integration into live healthcare platforms.

The key contributions of this project are:

- Effective handling of class imbalance using SMOTE
- Meaningful feature engineering to capture non-linear diabetes risk patterns
- Practical threshold tuning to optimize Recall for diabetes detection
- Model explainability through feature importance analysis
- Real-time diabetes prediction simulation ready for API deployment

6.1. Future Work

- Integration with deep learning models (LSTM, Autoencoders) for temporal health patterns
- Real-time API deployment using Flask or FastAPI
- Explainability improvements using SHAP (SHapley Additive exPlanations)
- Continuous model retraining with updated patient health data
- Incorporation of genetic and lifestyle data for enhanced prediction accuracy

7. Novelty

- Combines **SMOTE**, **feature engineering**, and **threshold tuning** for improved diabetes prediction.
- Focuses on **high Recall** to reduce missed diabetic cases in healthcare applications.
- Uses **custom engineered features** to capture hidden diabetes risk patterns.
- Includes a **real-time prediction simulation** for practical deployment.
- Provides **feature importance analysis** for better model interpretability.

8. REFERENCES

- [Kaggle Pima Indians Diabetes Dataset](#)
- [Scikit-learn Documentation](#)
- [Imbalanced-learn Documentation](#)
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research (JAIR), 16, 321–357.
- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd Edition). Morgan Kaufmann Publishers.
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd Edition). O'Reilly Media.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Aggarwal, C. C. (2015). *Data Mining: The Textbook*. Springer.
- [Pandas Documentation](#)
- [NumPy Documentation](#)
- [Matplotlib Documentation](#)
- [Seaborn Documentation](#)
- Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). *Calibrating Probability with Undersampling for Unbalanced Classification*. IEEE Symposium Series on Computational Intelligence.